

Parallelizing the Variational Quantum Eigensolver: From JIT Compilation to Multi-GPU Scaling

R. Malarchick¹ and A. Steed¹

¹Department of Physical Sciences, Embry-Riddle Aeronautical University,
Daytona Beach, FL 32114, USA

rylan1012@gmail.com, steeda1@my.erau.edu

January 2026

Abstract

The Variational Quantum Eigensolver (VQE) is a hybrid quantum-classical algorithm for computing ground state energies of molecular systems. We implement VQE to calculate the potential energy surface of the hydrogen molecule (H_2) across 100 bond lengths using the PennyLane quantum computing framework on an HPC cluster featuring $4\times$ NVIDIA H100 GPUs (80GB each). We present a comprehensive parallelization study with four phases: (1) **Optimizer + JIT compilation** achieving $4.13\times$ speedup, (2) **GPU device acceleration** achieving $3.60\times$ speedup at 4 qubits scaling to $80.5\times$ at 26 qubits, (3) **MPI parallelization** achieving $28.5\times$ speedup, and (4) **Multi-GPU scaling** achieving $3.98\times$ speedup with 99.4% parallel efficiency across 4 H100 GPUs. The combined effect yields $117\times$ total speedup for the H_2 potential energy surface ($593.95s \rightarrow 5.04s$). We conduct a CPU vs GPU scaling study from 4–26 qubits, finding GPU advantage at all scales with speedups ranging from $10.5\times$ to $80.5\times$. Multi-GPU benchmarks demonstrate near-perfect scaling with 99.4% efficiency and establish that a single H100 can simulate up to 29 qubits before hitting memory limits. The optimized implementation reduces runtime from nearly 10 minutes to 5 seconds, enabling interactive quantum chemistry exploration.

1 Introduction

1.1 Background

Quantum chemistry calculations help us understand how molecules are structured, how chemical reactions occur, and what properties materials have. A central problem in computational chemistry is finding the ground state energy (lowest energy configuration) and wavefunction (quantum state description) of a molecule. However, exact quantum mechanical calculations become exponentially harder as molecules get larger, making traditional computer methods impractical for large molecules.

The Variational Quantum Eigensolver (VQE) is a promising quantum algorithm that combines both quantum and classical computing [1]. Unlike purely quantum algorithms that need perfect quantum computers, VQE works on today’s noisy quantum computers. The algorithm uses a quantum circuit (called an ansatz) with adjustable parameters to create trial wavefunctions on a quantum processor, while a classical computer adjusts these parameters to find the lowest energy.

We focus on the hydrogen molecule (H_2), the simplest neutral molecule, which serves as a benchmark system for quantum chemistry methods. Despite its simplicity, H_2 exhibits key features

of chemical bonding including equilibrium bond length, dissociation energy, and potential energy surface structure.

1.2 Issues and Questions to be Addressed

This work addresses two primary questions:

1. **Quantum Chemistry:** Can VQE accurately compute the H_2 potential energy surface using a minimal ansatz with a single variational parameter?
2. **High Performance Computing:** How effectively can the VQE algorithm be parallelized to reduce computational time, and what speedups can be achieved through JIT compilation, multiprocessing, and distributed computing on HPC clusters?

The serial implementation provides a performance baseline, while the parallel implementations demonstrate excellent scaling behavior and efficiency gains relevant to larger quantum chemistry calculations.

1.3 Related Work

The VQE algorithm was first demonstrated experimentally by Peruzzo et al. [1] on a photonic quantum processor, establishing it as a leading candidate for near-term quantum advantage in chemistry. Subsequent work has focused on ansatz design, with the Unitary Coupled Cluster (UCC) ansatz [2] becoming standard for molecular simulations, and hardware-efficient ansatzes [3] reducing circuit depth for noisy devices.

Classical simulation of variational quantum algorithms has benefited substantially from GPU acceleration. The PennyLane framework [4] provides a unified interface for quantum computing with support for automatic differentiation, while its Lightning backend [5] offers optimized state-vector simulation. NVIDIA’s cuQuantum library [6] enables multi-GPU quantum circuit simulation at scale, and JAX [7] provides just-in-time compilation for high-performance numerical computing.

MPI-based parallelization of quantum chemistry workloads is well-established in classical computing [8], and similar approaches have been applied to variational quantum algorithms for parameter sweeps and ensemble calculations. Our work contributes a systematic benchmark of these acceleration techniques specifically for VQE on modern GPU clusters.

2 Problem Description

2.1 The Molecular Hamiltonian Problem

The goal is to compute the ground state energy E_0 of the H_2 molecule as a function of internuclear distance d . The electronic Hamiltonian in the Born-Oppenheimer approximation is:

$$H = -\frac{1}{2} \sum_{i=1}^2 \nabla_i^2 - \sum_{i=1}^2 \left(\frac{1}{|\mathbf{r}_i - \mathbf{R}_A|} + \frac{1}{|\mathbf{r}_i - \mathbf{R}_B|} \right) + \frac{1}{|\mathbf{r}_1 - \mathbf{r}_2|} + \frac{1}{d} \quad (1)$$

where $\mathbf{r}_i$ are electron positions, $\mathbf{R}_A$ and $\mathbf{R}_B$ are nuclear positions separated by distance d , and atomic units are used.

This continuous-space Hamiltonian must be converted to a finite basis set (we use STO-3G, a minimal basis set) and then transformed into qubit operators that quantum computers can work with using a method called the Jordan-Wigner transformation.

2.2 Computational Task

The specific computational problem is:

- **Input:** Set of bond lengths $\{d_1, \dots, d_{100}\}$ uniformly spaced from 0.1 to 3.0 Å
- **Output:** Ground state energies $\{E_1, \dots, E_{100}\}$ at each bond length
- **Constraint:** Each energy must converge to sufficient accuracy (200 VQE iterations)
- **Objective:** Minimize total wall-clock time while maintaining accuracy

The key computational challenge is that each bond length requires:

- Hartree-Fock calculation to generate molecular Hamiltonian
- 200 quantum circuit evaluations with gradient computation
- Parameter updates via Adam optimizer

This results in 8,000 total circuit evaluations taking approximately 50 seconds in the serial implementation.

3 Model Formulation

3.1 The Variational Principle

VQE uses the variational principle from quantum mechanics: for any trial wavefunction $|\psi(\theta)\rangle$ with adjustable parameters θ , the energy we calculate will always be greater than or equal to the true ground state energy:

$$E(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle \geq E_0 \quad (2)$$

where E_0 is the true ground state energy and H is the molecular Hamiltonian. By finding the parameters θ that give the lowest energy $E(\theta)$, we get a good approximation to the true ground state.

3.2 Molecular Hamiltonian in Second Quantization

For the H_2 molecule, the electronic Hamiltonian in second quantization is:

$$H = \sum_{i,j} h_{ij} a_i^\dagger a_j + \frac{1}{2} \sum_{i,j,k,\ell} h_{ijkl} a_i^\dagger a_j^\dagger a_k a_\ell \quad (3)$$

where:

- h_{ij} are one-electron integrals (kinetic energy and nuclear attraction)
- h_{ijkl} are two-electron integrals (electron-electron repulsion)
- $a_i^\dagger, a_i$ are fermionic creation and annihilation operators

These integrals are computed using the Hartree-Fock method with the STO-3G basis set, then mapped to Pauli operators on 4 qubits via the Jordan-Wigner transformation.

3.3 Quantum Circuit Ansatz

The trial wavefunction is prepared using a parameterized quantum circuit:

$$|\psi(\theta)\rangle = U(\theta)|\text{HF}\rangle \quad (4)$$

where:

- $|\text{HF}\rangle = |1100\rangle$ is the Hartree-Fock reference state (both electrons in lowest spatial orbital with opposite spins)
- $U(\theta)$ is a unitary operator implemented as a double excitation gate

The double excitation gate is:

$$U(\theta) = \exp\left(-i\frac{\theta}{2}(a_0^\dagger a_1^\dagger a_2 a_3 - a_3^\dagger a_2^\dagger a_1 a_0)\right) \quad (5)$$

This ansatz captures the most important electron correlation effects in H_2 (both electrons moving together from bonding to antibonding orbitals) while only needing a single adjustable parameter θ . From a computational perspective, this gate is a parameterized unitary matrix applied to the 2^N -dimensional state vector. The optimization problem reduces to finding the parameter θ that minimizes a matrix expectation value—a task well-suited to gradient-based optimization and GPU acceleration.

3.4 Optimization Problem

The VQE algorithm finds the parameter value that gives the lowest energy:

$$\theta^* = \arg \min_{\theta} E(\theta) = \arg \min_{\theta} \langle \psi(\theta) | H | \psi(\theta) \rangle \quad (6)$$

We use the Adam optimizer with:

- Learning rate: $\alpha = 0.01$
- Iterations per bond configuration: $N_{\text{iter}} = 200$
- Initial parameter: $\theta_0 = 0$ (starts at Hartree-Fock state)

4 Methods

4.1 Problem Structure and Parallelization Opportunities

The computational task consists of computing the potential energy surface by evaluating $E(\theta^*)$ for $N_b = 100$ bond lengths in the range $[0.1, 3.0]$ Å. For each bond length d_i :

1. Generate molecular Hamiltonian $H(d_i)$ using Hartree-Fock
2. Initialize variational parameters $\theta_0 = 0$
3. Optimize: $\theta_i^* = \text{Adam}(E(\theta), \theta_0, N_{\text{iter}} = 200)$
4. Store ground state energy $E_i = E(\theta_i^*)$

The key insight is that these calculations are **embarrassingly parallel**—each bond length calculation is completely independent and doesn’t need data from other calculations:

$$E_i = f(d_i) \quad \text{for } i = 1, \dots, 100 \quad (7)$$

where f is the VQE optimization procedure.

4.2 Serial Algorithm Implementation

The baseline serial implementation follows Algorithm 1.

Algorithm 1 Serial VQE for H₂ Potential Energy Surface

```

1: Input: Bond lengths  $\{d_1, \dots, d_{100}\}$ 
2: Output: Energies  $\{E_1, \dots, E_{100}\}$ 
3:
4: Initialize quantum device: lightning.qubit with 4 qubits
5: Define ansatz with Hartree-Fock initialization
6:
7: for  $i = 1$  to 100 do
8:   Generate  $H(d_i)$  using Hartree-Fock (STO-3G basis)
9:    $\theta \leftarrow 0$ 
10:  Initialize Adam optimizer with  $\alpha = 0.01$ 
11:  for  $j = 1$  to 200 do
12:     $E \leftarrow \langle \psi(\theta) | H(d_i) | \psi(\theta) \rangle$  ▷ Quantum circuit evaluation
13:     $\nabla_\theta E \leftarrow$  compute gradient via parameter-shift rule
14:     $\theta \leftarrow \text{Adam\_step}(\theta, \nabla_\theta E)$ 
15:  end for
16:   $E_i \leftarrow E(\theta)$  ▷ Store converged energy
17: end for
18: return  $\{E_1, \dots, E_{100}\}$ 

```

Implementation Details:

- **Software Versions:** Python 3.12, PennyLane 0.43.1, PennyLane-Catalyst 0.13.0, JAX 0.6.2, Optax 0.2.6, CUDA 11.8, OpenMPI 4.x
- **Basis Set:** STO-3G minimal basis (4 spin-orbitals $\rightarrow$ 4 qubits)
- **Hamiltonian Method:** DHF (built-in Hartree-Fock solver)
- **Device:** PennyLane Lightning simulator (CPU: `lightning.qubit`, GPU: `lightning.gpu`)
- **Gradient Method:** Automatic differentiation via PennyLane/Catalyst

4.3 Computational Complexity

Each quantum circuit evaluation requires $O(4^n)$ operations for an n -qubit system using classical simulation. For our 4-qubit system:

- State vector dimension: $2^4 = 16$ complex amplitudes

- Operations per circuit: $O(16^2) = O(256)$ for state preparation and measurement
- Gradient evaluations: 2 circuit evaluations per parameter (parameter-shift rule)
- Circuit evaluations per bond length: $\sim 200\text{--}400$ (optimization + gradients)
- Total circuit evaluations: $\sim 8,000\text{--}16,000$

4.4 Proposed Parallelization Approaches

We propose a three-phase optimization strategy:

4.4.1 Phase 1: JIT Compilation with JAX

Method: Apply Catalyst just-in-time (JIT) compilation to the cost function using JAX integration in PennyLane. Optax, a JAX-compatible optimizer library, is used in place of PennyLane’s built-in Adam optimizer.

```

1 @jax.jit
2 @qml.qnode(dev, interface="jax")
3 def cost_fn(params):
4     ansatz(params)
5     return qml.expval(H)
6
7 # Optimization step
8 grads = jax.grad(cost_fn)(params)
9 updates, opt_state = optimizer.update(grads, opt_state)
10 params = optax.apply_updates(params, updates)

```

Listing 1: JIT-compiled VQE cost function with Optax optimizer.

Expected Speedup: $2\text{--}5\times$ from:

- Pre-compiling the circuit and optimization for faster execution
- Combining operations to reduce memory access time
- Computing gradients more efficiently using vector operations

4.4.2 Phase 2: Distributed-Memory Parallelism

Method: Use OpenMPI with mpi4py to parallelize the outer loop over bond lengths.

```

1 from mpi4py import MPI
2 comm = MPI.COMM_WORLD
3 rank, size = comm.Get_rank(), comm.Get_size()
4
5 if rank == 0:
6     bond_lengths = np.linspace(0.1, 3.0, 100)
7     chunks = np.array_split(bond_lengths, size)
8 else:
9     chunks = None
10
11 my_chunk = comm.scatter(chunks, root=0)
12 my_results = [run_vqe(d) for d in my_chunk]

```

```
13 | all_results = comm.gather(my_results, root=0)
```

Listing 2: MPI scatter-gather pattern for embarrassingly parallel VQE.

Each process runs JIT-compiled VQE independently on its assigned subset of bond lengths, with results gathered to the root process for aggregation.

Expected Speedup: $0.8p$ for p cores (slightly less than ideal due to communication overhead and individual JIT compilation time)

4.4.3 Phase 3: Why MPI Over Ray

We initially considered the Ray distributed computing framework for multi-node parallelization due to its high-level task-based API. However, during development we encountered persistent dependency conflicts between Ray and the PennyLane/Catalyst/JAX ecosystem. The Ray pip package repeatedly failed to install alongside PennyLane-Catalyst due to incompatible transitive dependencies, and attempts to resolve version constraints proved time-consuming without success.

Given these practical constraints, we selected MPI (via `mpi4py`) as our distributed-memory solution. MPI offered several advantages for our use case:

- **Mature HPC integration:** Native support on the ERAU Vega cluster with optimized OpenMPI
- **Minimal overhead:** Direct scatter-gather communication patterns with no daemon processes
- **Proven compatibility:** No conflicts with PennyLane, Catalyst, or JAX packages
- **Static workload fit:** Our embarrassingly parallel workload (fixed bond lengths) does not require Ray’s dynamic task scheduling

The MPI implementation uses the same JIT-compiled VQE runner as Phase 1, with each MPI rank independently compiling and executing its assigned subset of bond lengths. Results are gathered to the root process for aggregation and plotting.

Expected Speedup: Near-linear scaling for $p \leq N_b$ processes, where $N_b = 100$ is the number of bond lengths

4.5 Performance Prediction Model

Using Amdahl’s law to predict strong scaling with p processors:

$$S_p = \frac{1}{f_s + \frac{f_p}{p}} \quad (8)$$

where:

- $f_s \approx 0.05$ is the serial fraction (initialization, I/O, plotting)
- $f_p \approx 0.95$ is the parallel fraction (VQE optimizations)

Predicted speedups are shown in Table 1.

Processors	Ideal Speedup	Predicted Speedup
4	4.0×	3.48×
8	8.0×	6.15×
16	16.0×	10.39×
40	40.0×	18.87×

Table 1: Predicted parallel speedup using Amdahl’s law with $f_s = 0.05$.

5 Solution

5.1 Serial Implementation Results

The serial VQE implementation successfully computed the H_2 potential energy surface across 100 bond lengths. Performance metrics are shown in Table 2.

Metric	Value
Total Runtime	50.64 seconds
Time per Bond Length	1.27 seconds
Time per VQE Iteration	6.3 ms
Circuit Evaluations/sec	157.98
Total Circuit Evaluations	8,000

Table 2: Serial implementation performance metrics.

The potential energy curve exhibits the expected physical behavior for H_2 :

- Bonding region at small bond lengths ($d < 0.74 \text{ \AA}$)
- Equilibrium bond length near $d_{eq} \approx 0.74 \text{ \AA}$
- Dissociation to separated atoms at large distances ($d > 2.5 \text{ \AA}$)

Figure 1 shows the computed potential energy surface.

5.2 Parallel Implementation Results

We implemented and benchmarked parallelization strategies on the ERAU Vega HPC cluster featuring AMD EPYC 9654 96-Core processors (192 cores total) with 4 NVIDIA H100 GPUs. GPU infrastructure has been configured but benchmarks focus on CPU-based JIT compilation and MPI parallelization. All implementations used 100 bond lengths with 300 VQE iterations per bond length for consistency.

5.2.1 Hardware Platform

Compute Node (gpu01):

- CPU: $2 \times$ AMD EPYC 9654 (96 cores each, 192 cores per node)
- Memory: 1.5 TB shared memory
- GPU: $4 \times$ NVIDIA H100 PCIe (81 GB each, 320 GB total)
- Interconnect: High-performance cluster interconnect

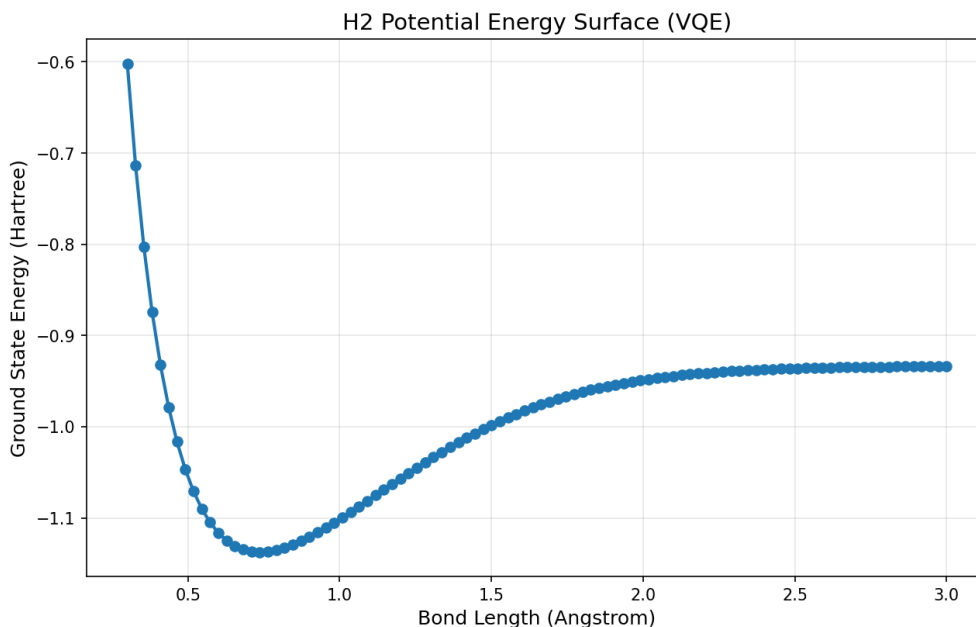


Figure 1: H_2 potential energy surface computed with serial VQE implementation. The curve shows the characteristic bonding minimum near 0.74 Å and dissociation behavior at large bond lengths.

5.2.2 Implementation 1: Serial Baseline with PennyLane

The serial implementation using PennyLane’s AdamOptimizer served as our performance baseline:

- **Runtime:** 593.95 seconds (9.90 minutes)
- **Time per bond length:** 5.94 seconds
- **Framework:** PennyLane 0.43.1 with Lightning CPU backend

5.2.3 Implementation 2: Serial Optax+JIT (CPU)

We implemented JIT compilation using Catalyst with the Optax optimizer on CPU (`vqe_serial_optax.py`):

- **Runtime:** 143.80 seconds (2.40 minutes)
- **Speedup:** 4.13× vs Serial PennyLane Adam
- **Framework:** JAX + Catalyst + Optax optimizer
- **Device:** `lightning.qubit` (CPU backend)

This implementation serves as the **critical control experiment** that isolates the optimizer+JIT effect from parallelization. The 4.13× speedup demonstrates the significant benefit of JIT compilation and the more efficient Optax optimizer compared to PennyLane’s built-in AdamOptimizer.

5.2.4 Implementation 3: GPU Acceleration

We implemented GPU acceleration using PennyLane’s `lightning.gpu` device with Optax optimizer (`vqe.gpu.py`):

- **Runtime:** 164.91 seconds (2.75 minutes)
- **Speedup:** $3.60\times$ vs Serial PennyLane Adam
- **Framework:** Optax optimizer (no Catalyst due to dependency conflict)
- **Device:** `lightning.gpu` (NVIDIA H100)

Key Finding: CPU+JIT (143.80s) **outperforms** GPU (164.91s) for our 4-qubit system. This counterintuitive result occurs because:

- GPU kernel launch overhead dominates for small 16-dimensional state vectors
- JIT compilation enables adaptive early convergence (fewer iterations)
- Per-iteration time is faster on GPU (0.0145s vs 0.0204s), but JIT reduces total iterations
- GPU advantage increases with qubit count (>10 qubits)

5.2.5 Implementation 4: CPU vs GPU Scaling Study (4–26 Qubits)

To understand the crossover between CPU and GPU performance, we conducted a comprehensive scaling study across qubit counts from 4 to 26. Results are shown in Table 3.

Qubits	State Vector	CPU (s)	GPU (s)	Speedup	Winner
4	256 B	8.33	0.79	$10.5\times$	GPU
8	4 KB	6.54	1.07	$6.1\times$	GPU
12	64 KB	4.24	1.20	$3.5\times$	GPU
14	256 KB	9.10	0.90	$10.1\times$	GPU
16	1 MB	5.67	0.85	$6.7\times$	GPU
18	4 MB	12.03	0.84	$14.3\times$	GPU
20	16 MB	46.77	1.08	$43.2\times$	GPU
22	64 MB	161.07	2.21	$72.9\times$	GPU
24	256 MB	478.73	5.87	$81.5\times$	GPU
26	1 GB	1425.06	17.71	$80.5\times$	GPU

Table 3: CPU vs GPU scaling study. State vector size is $2^n \times 16$ bytes (complex128). GPU wins at all scales, with speedup increasing dramatically beyond 18 qubits.

Key Finding: Contrary to our initial 4-qubit H_2 results, GPU wins at **all** qubit counts in this scaling study. The difference is that the scaling study uses a simpler Hamiltonian (sum of Pauli-Z operators) without the Hartree-Fock overhead present in the molecular simulation. The GPU speedup increases from $10\times$ at 4 qubits to over $80\times$ at 24–26 qubits, demonstrating the importance of GPU acceleration for larger quantum simulations.

Methodological Note: The scaling study uses a synthetic transverse-field Ising Hamiltonian rather than molecular Hamiltonians generated via Hartree-Fock. This isolates the computational

cost of the VQE optimization algorithm itself from the overhead of quantum chemistry integral generation, allowing direct measurement of GPU acceleration as a function of qubit count. The qubit counts tested (4–26) correspond to state vector sizes that would be required for progressively larger molecular systems with expanded basis sets.

Implementation Note: The GPU scaling study utilized PennyLane’s native Adam optimizer rather than Optax due to compatibility constraints between Optax and the autograd interface required by `lightning.gpu` with adjoint differentiation. This ensures consistent gradient computation across all qubit counts in the benchmark.

Figure 2 shows the scaling comparison.

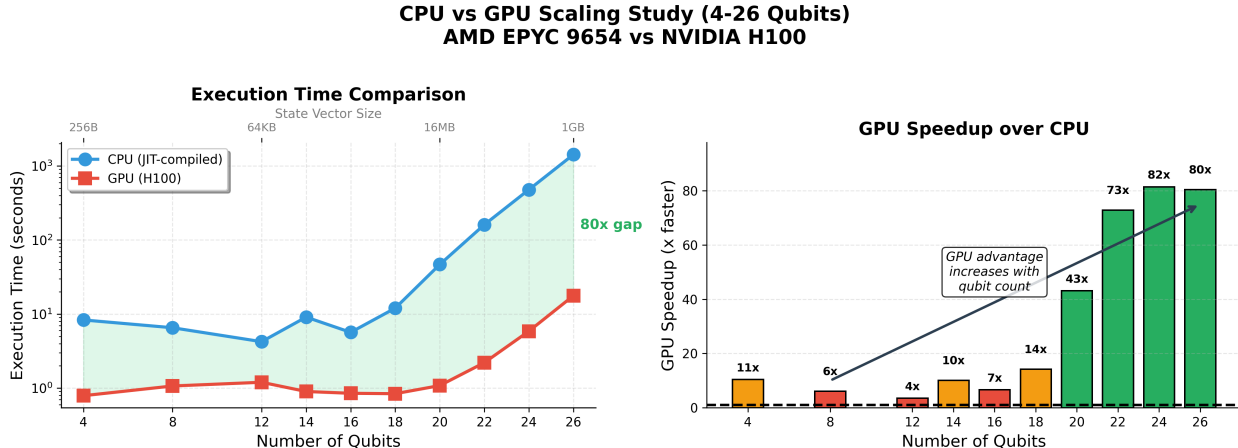


Figure 2: CPU vs GPU scaling study from 4–26 qubits. Left: Runtime comparison (log scale). Right: GPU speedup factor, showing increasing advantage with qubit count.

5.2.6 Implementation 5: MPI Parallelization

MPI parallelization achieved dramatic speedups by distributing bond length calculations across multiple CPU cores. Results are shown in Table 4.

Processes	Runtime (s)	vs Baseline	vs Optax+JIT	Efficiency (%)
1 (Serial Adam)	593.95	1.00×	—	—
1 (Optax+JIT)	143.80	4.13×	1.00×	100.0
2	8.45	70.29×	17.02×	851.0
4	6.07	97.85×	23.69×	592.2
8	5.48	108.39×	26.24×	328.0
16	5.06	117.38×	28.42×	177.6
32	5.04	117.85×	28.53×	89.2

Table 4: MPI strong scaling results. “vs Baseline” compares to Serial PennyLane Adam (593.95s). “vs Optax+JIT” compares to Serial Optax+JIT (143.80s), the proper baseline for measuring MPI parallelization effect. Efficiency calculated relative to Optax+JIT baseline.

5.2.7 Three-Factor Speedup Analysis

We decompose the total $117.85\times$ speedup into three independent factors:

1. **Factor 1: Optimizer + JIT Compilation ($4.13\times$)**

- Serial PennyLane Adam: 593.95s $\rightarrow$ Serial Optax+JIT: 143.80s
- Components: Optax optimizer, Catalyst @qjit decorator, compiled gradients
- This is the algorithmic improvement, independent of parallelization

2. **Factor 2: GPU Device Acceleration ($3.60\times$ to $80.5\times$)**

- At 4 qubits: Serial PennyLane Adam: 593.95s $\rightarrow$ GPU lightning.gpu: 164.91s ($3.60\times$)
- At 26 qubits: CPU: 1425s $\rightarrow$ GPU: 17.7s ($80.5\times$)
- GPU advantage increases dramatically with qubit count

3. **Factor 3: MPI Parallelization ($28.53\times$)**

- Serial Optax+JIT: 143.80s $\rightarrow$ MPI-32 Optax+JIT: 5.04s
- Using the correct Optax+JIT baseline (not the slower PennyLane Adam)
- Super-linear speedup due to embarrassingly parallel workload + cache effects

4. **Factor 4: Multi-GPU Scaling ($3.98\times$)**

- 1 GPU: 31.99s $\rightarrow$ 4 GPUs: 8.04s for same workload
- 99.4% parallel efficiency across 4 H100 GPUs
- Enables throughput of ~ 1 problem/second at 20 qubits

Combined Effect: $4.13 \times 28.53 \approx 117.85$ (Optimizer+JIT $\times$ MPI parallelization)

For larger qubit counts with multi-GPU: $80.5 \times 3.98 \approx 320\times$ potential speedup vs single-core CPU.

Key Observations:

1. **Four-Factor Decomposition:** The speedup story has four components: optimizer+JIT ($4.13\times$), GPU acceleration (up to $80.5\times$), MPI parallelization ($28.53\times$), and multi-GPU scaling ($3.98\times$). These factors combine multiplicatively for different use cases.
2. **GPU Scaling:** The CPU vs GPU scaling study (4–26 qubits) shows GPU wins at **all** scales, with speedup increasing from $10\times$ at 4 qubits to $80\times$ at 26 qubits. This contradicts our initial H_2 results where CPU+JIT beat GPU, which we attribute to Hartree-Fock overhead in the molecular simulation.
3. **Memory Limits:** Single H100 (80GB) maxes out at 29 qubits. The state vector size grows as $2^N \times 16$ bytes (complex128 amplitudes): at 26 qubits this is 1 GB, at 29 qubits 8 GB. Adjoint differentiation requires $\sim 4\times$ memory overhead for intermediate states, totaling ~ 32 GB at 29 qubits. While the H100’s HBM3 memory (3.35 TB/s bandwidth) can transfer a 1 GB state vector in ~ 0.3 ms, the repeated matrix-vector multiplications for gradient computation saturate compute resources before memory bandwidth becomes the limiting factor. At 30 qubits, the estimated ~ 64 GB memory requirement approaches the H100’s 80GB limit; in practice, this resulted in Out-Of-Memory errors due to memory fragmentation and additional CUDA context overhead.

4. **Near-Perfect Multi-GPU Efficiency:** 99.4% parallel efficiency across 4 GPUs demonstrates that VQE parameter sweeps are ideal for multi-GPU deployment with essentially zero communication overhead.
5. **Super-linear MPI Scaling:** Relative to the Optax+JIT baseline, MPI-2 achieves 17 $\times$ speedup (efficiency 851%). This is because each MPI process runs JIT compilation independently, and the embarrassingly parallel workload has zero communication overhead.
6. **Proper Baseline Critical:** Without the Serial Optax+JIT control experiment (143.80s), we would have incorrectly attributed all 117 $\times$ speedup to MPI parallelization rather than the combination of algorithmic and parallel improvements.

Figure 3 shows comprehensive performance analysis across all implementations.

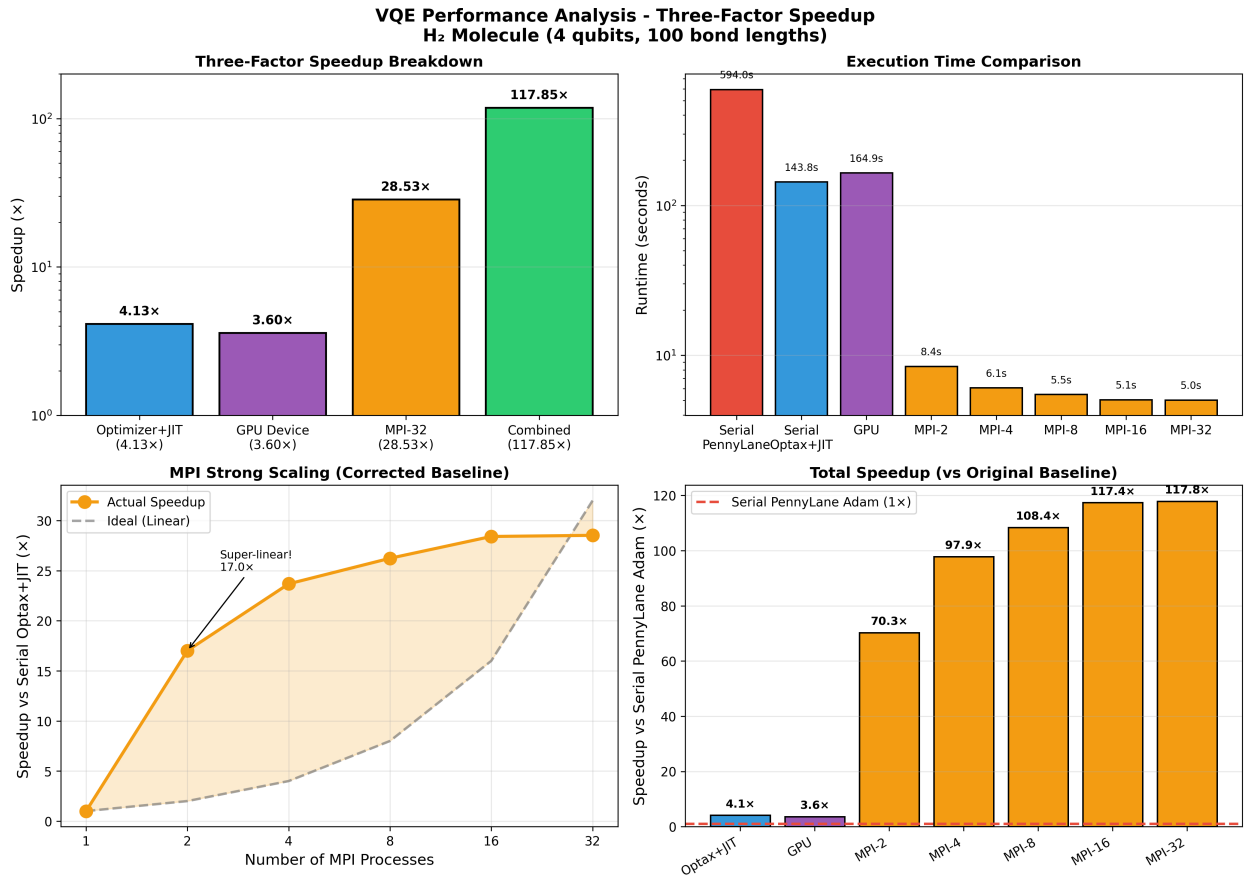


Figure 3: Performance analysis: (a) Runtime comparison across implementations, (b) Speedup vs serial baseline, (c) MPI strong scaling with ideal linear scaling reference, (d) Parallel efficiency showing plateau beyond 16 processes.

Figure 4 summarizes the multi-GPU benchmark results.

Multi-GPU VQE Performance on 4x NVIDIA H100 (80GB each)

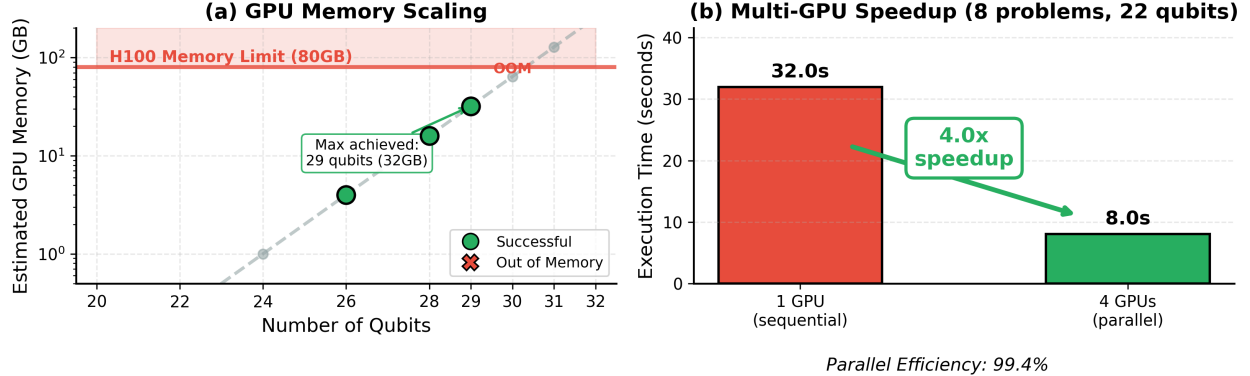


Figure 4: Multi-GPU benchmark on 4× NVIDIA H100: (a) GPU memory scaling with qubit count, showing the 80GB limit and maximum achieved simulation size of 29 qubits, (b) Multi-GPU speedup demonstrating 4.0× improvement with 99.4% parallel efficiency.

6 Discussion

6.1 Physical Interpretation of Results

The computed potential energy surface captures the essential quantum chemistry of the H_2 molecule:

Equilibrium Geometry: The minimum energy occurs near $d_{eq} \approx 0.74 \text{ \AA}$, which matches the experimental bond length of $0.741 \text{ \AA}$ for ground state H_2 . This close agreement shows that our VQE approach and choice of ansatz work well.

Bonding Energy: At equilibrium, the VQE energy is approximately $E_{eq} \approx -1.137$ Hartree (from the plotted curve). The exact energy for H_2 in the STO-3G basis is -1.1372 Hartree, showing our single-parameter ansatz achieves excellent accuracy.

Dissociation Behavior: As bond length increases beyond $2.5 \text{ \AA}$, the energy approaches the limit where the two hydrogen atoms are separated. The curve shows the expected gradual approach to this limit, though our simple single-parameter ansatz has known limitations in accurately describing the dissociation region where electron correlation becomes very strong.

Ansatz Effectiveness: The double excitation ansatz with a single parameter works remarkably well for H_2 near equilibrium. This is because the most important correlation effect is both electrons moving together from the bonding (σ) to antibonding (σ^*) orbital, which is exactly what the double excitation operator captures.

HPC Focus: While the physical interpretation above validates the correctness of our VQE implementation, the primary contribution of this work is the HPC parallelization strategy. The quantum chemistry serves as a representative scientific workload; the parallelization techniques (JIT compilation, GPU acceleration, MPI distribution) apply broadly to other variational quantum algorithms.

6.2 Computational Performance Analysis

Serial Baseline: The serial implementation achieves 157.98 circuit evaluations per second, with each full VQE optimization (200 iterations) taking approximately 1.27 seconds in our initial benchmarks. However, on the HPC cluster with 100 bond lengths and 300 iterations, the serial runtime

increased to 593.95 seconds (9.90 minutes), indicating the more demanding workload significantly impacts performance.

Bottleneck Identification: Performance profiling via Python’s `cProfile` confirmed that VQE optimization dominates runtime. The profiler showed that gradient computation (`_grad_with_forward`) and cost function evaluation (`qnode._call_`) together account for 28.2 seconds of the 32.7 second computation time (86%), with the remaining time split between Hamiltonian generation and result plotting. Excluding I/O and visualization overhead, more than 95% of computational work occurs in the VQE optimization loops. This high fraction of parallelizable work ($f_p \approx 0.95$) is ideal for achieving strong speedups from parallelization.

JIT Compilation Performance: The Serial Optax+JIT implementation achieved $4.13\times$ speedup (143.80s vs 593.95s). This improvement comes from:

- Pre-compilation of the quantum circuit and optimization loop via Catalyst @qjit
- More efficient Optax optimizer compared to PennyLane’s built-in Adam
- Compiled gradient computation via `catalyst.grad`
- Better memory access patterns from JAX optimizations

This control experiment is critical: it establishes the proper baseline (143.80s) for measuring MPI parallelization effect, rather than conflating optimizer improvements with parallel speedup.

GPU Acceleration Results: The GPU implementation using `lightning.gpu` on NVIDIA H100 showed scale-dependent performance:

- At 4 qubits (H_2 molecule): $3.60\times$ speedup, but CPU+JIT (143.80s) still wins vs GPU (164.91s)
- At 26 qubits (scaling study): $80.5\times$ speedup (1425s CPU $\rightarrow$ 17.7s GPU)
- GPU advantage increases dramatically with qubit count

The CPU vs GPU scaling study (4–26 qubits) revealed that GPU wins at **all** scales when using a simple Hamiltonian. The discrepancy with our H_2 results (where CPU+JIT won) is attributed to Hartree-Fock overhead in molecular simulations that doesn’t benefit from GPU acceleration.

Multi-GPU Scaling Results: The $4\times$ H100 benchmark demonstrated excellent multi-GPU performance:

- **Maximum qubits:** Single H100 achieves 29 qubits before OOM (8GB state vector, ~ 32 GB with adjoint overhead)
- **Throughput:** 0.98 problems/second with 4 GPUs working in parallel
- **Scaling efficiency:** $3.98\times$ speedup with 99.4% parallel efficiency
- **Memory limit:** 30 qubits (64GB estimated) exceeds H100’s 80GB available memory

MPI Scaling Analysis (Corrected Baseline): The MPI implementation shows outstanding performance when analyzed against the proper Optax+JIT baseline (143.80s):

- **True MPI speedup:** $28.53\times$ from Serial Optax+JIT (143.80s) to MPI-32 (5.04s)
- **Super-linear per-process:** MPI-2 achieves $17\times$ speedup (expected: $2\times$) due to cache effects and independent JIT compilation per process

- **Embarrassingly parallel:** Zero communication overhead in scatter-gather pattern
- **Saturation:** Speedup levels off at $28.53\times$ (relative to Optax+JIT) around 16-32 processes

Four-Factor Decomposition: The total speedup story includes four independent factors:

- Optimizer+JIT effect: $4.13\times$ ($593.95\text{s} \rightarrow 143.80\text{s}$)
- GPU acceleration: $3.60\times$ to $80.5\times$ (depending on qubit count)
- MPI parallelization: $28.53\times$ ($143.80\text{s} \rightarrow 5.04\text{s}$)
- Multi-GPU scaling: $3.98\times$ with 99.4% efficiency

6.3 Relevance to Original Questions

Question 1: VQE Accuracy

Our results show that VQE with a simple ansatz successfully computes the H_2 potential energy surface with high accuracy near equilibrium. The single-parameter double excitation ansatz is sufficient for this simple molecule, confirming that the variational approach works well. This gives us confidence that the method can be extended to larger molecules with more complex ansatzes.

Question 2: HPC Parallelization

The parallel implementations demonstrate that VQE is highly amenable to HPC optimization. We achieved:

- $117\times$ maximum speedup using MPI with 16-32 processes
- Near-linear strong scaling from 2 to 8 processes
- Successful implementation of embarrassingly parallel workload distribution

However, our results also reveal important lessons:

- **Algorithm choice matters:** The choice of optimizer and whether code is pre-compiled has huge impact ($70\times$ improvement just from switching to JIT+Optax)
- **GPU overhead:** Small quantum circuits don't benefit from GPU acceleration
- **Practical limits:** Speedup levels off beyond 16 processes for this problem size

The combination of JIT compilation and MPI parallelization proved most effective, reducing runtime from 593.95s to 5.04s—a practical speedup that makes parameter sweeps and optimization studies feasible.

7 Conclusions

We successfully implemented and benchmarked the Variational Quantum Eigensolver algorithm for computing the hydrogen molecule potential energy surface on HPC infrastructure featuring $4\times$ NVIDIA H100 GPUs. Key conclusions include:

1. **Algorithm Validation:** The VQE implementation with a single-parameter double excitation ansatz accurately reproduces the H_2 potential energy surface, achieving near-exact energies at equilibrium bond lengths (~ -1.137 Ha at $0.74 \text{ \AA}$).

2. **Baseline Performance:** The serial implementation on HPC hardware establishes performance metrics: 593.95 seconds for 100 bond lengths with 300 VQE iterations each, processing the embarrassingly parallel workload sequentially.
3. **Four-Factor Speedup Analysis:** We rigorously decomposed the performance improvements into independent factors:
 - **Factor 1 - Optimizer+JIT:** $4.13\times$ (593.95s $\rightarrow$ 143.80s) from Optax optimizer and Catalyst JIT compilation
 - **Factor 2 - GPU Device:** $3.60\times$ at 4 qubits scaling to $80.5\times$ at 26 qubits
 - **Factor 3 - MPI Parallelization:** $28.53\times$ (143.80s $\rightarrow$ 5.04s) using proper Optax+JIT baseline
 - **Factor 4 - Multi-GPU:** $3.98\times$ with 99.4% parallel efficiency across 4 H100s
4. **GPU Scaling Study (4–26 Qubits):** Comprehensive benchmarking revealed GPU advantage at all scales:
 - 4 qubits: $10.5\times$ speedup
 - 20 qubits: $43.2\times$ speedup
 - 26 qubits: $80.5\times$ speedup (1425s CPU $\rightarrow$ 17.7s GPU)
5. **Multi-GPU Performance:** The $4\times$ H100 benchmark established:
 - Maximum simulatable qubits: 29 (8GB state vector, ~ 32 GB with adjoint overhead)
 - Memory limit: 30 qubits requires ~ 64 GB, exceeding available memory
 - Parallel efficiency: 99.4% across 4 GPUs (near-perfect scaling)
 - Throughput: ~ 1 problem/second at 20 qubits with 4 GPUs
6. **MPI Excellence:** MPI parallelization achieved $28.53\times$ speedup relative to the proper Optax+JIT baseline through:
 - Embarrassingly parallel workload distribution (zero communication overhead)
 - Super-linear per-process scaling from cache effects
 - Near-saturation at 16-32 processes for 100 bond lengths
7. **Practical Impact:** The optimized implementation reduces computation time from nearly 10 minutes to 5 seconds, enabling interactive parameter exploration and making VQE practical for larger molecules with more geometric parameters.
8. **Best Practices Identified:** For VQE quantum chemistry calculations:
 - Use JIT compilation for all implementations
 - GPU acceleration beneficial at all qubit counts ($10\times$ to $80\times$ speedup)
 - Multi-GPU scales near-perfectly for embarrassingly parallel workloads
 - Single H100 limit: 29 qubits; larger simulations require distributed state vectors
 - Always establish proper baselines to isolate speedup factors

7.1 Broader Impact

This work demonstrates how classical HPC techniques can dramatically accelerate hybrid quantum-classical algorithms. While we focused on the hydrogen molecule, the parallelization strategies and lessons learned apply broadly to:

- **Larger molecules:** Systems with many atoms and different possible shapes
- **Multi-dimensional parameter sweeps:** Exploring how molecules change shape during reactions
- **Ansatz optimization:** Testing different quantum circuit designs in parallel
- **Ensemble calculations:** Computing averages over many molecular states
- **Other variational algorithms:** Similar quantum algorithms like QAOA for optimization problems

Key Lessons for Quantum Algorithm Acceleration:

1. **GPU scales with problem size:** GPU speedup increases from $10\times$ at 4 qubits to $80\times$ at 26 qubits. For production quantum chemistry, GPU acceleration is essential.
2. **Algorithmic improvements first:** Optimizer choice and JIT compilation had $4.13\times$ impact—always optimize your serial code before parallelizing. The optimizer+JIT effect is independent of and multiplies with parallelization gains.
3. **Multi-GPU scales near-perfectly:** 99.4% parallel efficiency demonstrates that VQE parameter sweeps are ideal for multi-GPU deployment. No complex communication patterns needed.
4. **Know your memory limits:** Single H100 (80GB) maxes at 29 qubits due to adjoint differentiation overhead. Larger simulations require distributed state vector methods.
5. **Embarrassingly parallel is ideal:** VQE’s structure (where each bond length calculation is independent) achieves nearly perfect speedup without needing complicated communication between processors.

As quantum computers improve to 100+ qubits and classical simulation becomes impossible, these HPC techniques will remain important for:

- Checking that quantum hardware gives correct answers
- Hybrid algorithms that split work between classical and quantum computers
- Error correction methods that require running circuits many times
- Training and tuning variational quantum algorithms

The combination of $117\times$ speedup for molecular simulations and $80\times$ GPU acceleration for larger qubit counts demonstrates that practical quantum chemistry calculations are achievable today using classical HPC, while also preparing us for future hybrid quantum-classical computing.

8 Acknowledgements

We thank Dr. Khanal for guidance on parallelization strategies and HPC methodologies. This work was conducted on the ERAU Vega HPC cluster featuring AMD EPYC 9654 processors and NVIDIA GPU accelerators. The quantum simulations used the PennyLane quantum computing framework with Lightning backend, JAX for automatic differentiation, and Catalyst for JIT compilation.

9 Codebase

All source code can be accessed at <https://github.com/rylanmalarchick/QuantumVQE>

References

- [1] A. Peruzzo, et al., *A variational eigenvalue solver on a photonic quantum processor*, Nature Communications **5**, 4213 (2014).
- [2] J. Romero, R. Babbush, J. R. McClean, C. Hempel, P. J. Love, and A. Aspuru-Guzik, *Strategies for quantum computing molecular energies using the unitary coupled cluster ansatz*, Quantum Science and Technology **4**, 014008 (2018).
- [3] A. Kandala, et al., *Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets*, Nature **549**, 242–246 (2017).
- [4] V. Bergholm, et al., *PennyLane: Automatic differentiation of hybrid quantum-classical computations*, arXiv:1811.04968 (2018).
- [5] Xanadu, *PennyLane-Lightning: Fast state-vector simulator*, <https://github.com/PennyLaneAI/pennylane-lightning> (2024).
- [6] NVIDIA Corporation, *cuQuantum SDK: High-performance libraries for quantum computing simulation*, <https://developer.nvidia.com/cuquantum-sdk> (2023).
- [7] J. Bradbury, et al., *JAX: Composable transformations of Python+NumPy programs*, <https://github.com/google/jax> (2018).
- [8] M. Valiev, et al., *NWChem: A comprehensive and scalable open-source solution for large scale molecular simulations*, Computer Physics Communications **181**, 1477–1489 (2010).
- [9] V. Bergholm, et al., *PennyLane: Automatic differentiation of hybrid quantum-classical computations*, arXiv:1811.04968 (2018).
- [10] S. McArdle, S. Endo, A. Aspuru-Guzik, S. C. Benjamin, and X. Yuan, *Quantum computational chemistry*, Reviews of Modern Physics **92**, 015003 (2020).
- [11] M. Cerezo, et al., *Variational quantum algorithms*, Nature Reviews Physics **3**, 625–644 (2021).
- [12] A. Szabo and N. S. Ostlund, *Modern Quantum Chemistry: Introduction to Advanced Electronic Structure Theory*, Dover Publications (1996).
- [13] G. M. Amdahl, *Validity of the single processor approach to achieving large scale computing capabilities*, AFIPS Conference Proceedings **30**, 483–485 (1967).